



The Flaky Network (120 minutes)

The problem

A source produces one 160-byte frame every 20 milliseconds - think live audio. Frames must cross a network and be "played" on a steady 20 ms clock at the other end. The network between is flaky: it drops packets, delays them by random amounts, reorders them, and sometimes delivers them twice.

You write the two programs at either edge - the sender and the receiver - in **C, C++, Golang or Rust**. Make playout survive the network with as little added delay as possible. This is the tradeoff every real-time system on the internet lives on.

Architecture (all on 127.0.0.1, all UDP)

None

```
harness source --47010--> YOUR SENDER --47001--> relay --47002-->
YOUR RECEIVER --47020--> harness player
                        ^
|
                        47004 <----- relay <-----
47003                (feedback path, optional)
```

- The harness **source** hands your sender frame i at exactly $t_0 + i \cdot 20\text{ms}$. You cannot send a frame early; you don't have it yet.
- The **relay** is the hostile network (drops, jitter, reordering, duplicates). Media and feedback both pass through it.
- The harness **player** judges frame i at deadline $t_0 + \text{DELAY_MS} + i \cdot 20\text{ms}$: it must have arrived on port 47020, correct and on time, or it is a miss.
- Harness legs (47010 in, 47020 out) use a fixed format: **4-byte big-endian unsigned seq + 160-byte payload**. Between your sender and receiver (47001/47002/47003/47004) the wire format is entirely your design.

Your files (systems_handout.zip)



None

```
run.py, relay.py, endpoints.py, common.py, score.py    harness – DO NOT MODIFY
sender.c, receiver.c, Makefile                        naive C
baselines – YOURS
profiles/A.json, profiles/B.json                      practice
network conditions
```

Language: **C, C++, Golang or Rust** - your choice. The C baselines compile as C++; port to Rust if you prefer. Your submission must build with `make` into `./sender` and `./receiver` (a Makefile that shells out to cargo is fine).

Requirements: Linux, macOS, or WSL; a C/C++ compiler or Rust toolchain; Python 3.10+ for the harness. Run the baseline right now:

None

```
make && python3 run.py --profile profiles/A.json --delay_ms 40
```

It will be INVALID - every dropped packet is a glitch. Your hour starts there.

The score

- **Deadline-miss rate** must be $\leq 1\%$ for a run to count.
- **Bandwidth overhead** - every byte through the relay, both directions, including packets it then drops - must be $\leq 2.0\times$ the raw stream.
- **Playout delay** (`--delay_ms`) - among valid runs, **lowest wins**.

We grade with the same harness on **network profiles you have not seen** (different loss patterns, different jitter behavior). Tune for the mechanism, not for profile A. Your processes receive `T0`, `DURATION_S`, `DELAY_MS` as environment variables and are killed when the run ends.

Things worth thinking about (not a checklist)

1. What does a lost packet cost at 20 ms per frame, and what are the only two families of remedies?
2. What does a resend cost in time when the request AND the reply each cross the hostile network?
3. What could you send *ahead of time* instead, and what does the $2\times$ budget buy?



4. How big must your jitter buffer be for the delay spikes you actually measured - and what happens to every strategy when losses arrive in bursts?
5. And since you're in C/C++/ Rust: who owns each buffer, what's in network byte order, and does your receive loop ever block at the wrong moment?

Deliverables

1. Source for your sender and receiver + Makefile (`make` → `./sender`, `./receiver`)
2. `RUNLOG.md` - per experiment: profile, `delay_ms`, miss %, overhead, what you changed and why. **Graded.**
3. `NOTES.md` - max 10 sentences: your design, the `delay_ms` we should grade at, and what breaks it.
4. `SUMMARY.html` - Created by your coding agent that summarizes the architecture and the design choices you took, and explains the architecture in detail in terms of what was done and going to the system level.

Rules

- Standard library of your language only (sockets, threads, time). No networking frameworks.
- No side channels: your two processes talk ONLY through the relay ports.
- Harness files are replaced with our copies at grading; modifying them does nothing.
- AI coding assistants allowed. We know their unassisted score; your grade lives above it and in a 5-minute discussion after.

Suggested hour

- **0–10:** build, run baseline on A and B, read the numbers. Understand WHY it fails before typing.
- **10–40:** fix loss first (get VALID at a generous delay), then shrink delay. One change per run (~35 s each); log everything.
- **40–55:** stress on B, find your lowest valid delay, lock it.
- **55–60:** `RUNLOG.md`, `NOTES.md`, state your `delay_ms`.